TECHNICAL REPORT — THE VERTICAL AGGREGATION ARCHITECTURE

Nano-robots, swarm, meta-swarm

EFE’s three scales and how information flows **upward**: from a nano-agent’s tensor row, to a swarm’s centroid, to the meta-consensus of a swarm of swarms — with the **real** maturity of every stage.

Program	EFE → Kovant · proof dossier (the “dowry”)
Scope	efe Rust monorepo · HEAD 665ead0 (+ working tree) · efe-core kernel + meta-swarm
Date	July 14, 2026
Author	R. Pekar — with Claude Code (Opus 4.8)
Method	surgical code reading (file:line citations) + a dedicated honesty review
Cardinal rule	NEVER OVER-SELL — a nano-agent is a tensor row, not an autonomous robot

The right word, up front

The code says **nano-agent**; “nano-robot” is presentation vocabulary. At each scale, the “agent” and the “information exchanged” change nature: a nano-agent is a **row** of a latent tensor (its voice is a vector); a swarm is that population folded into **one centroid**; a meta-swarm couples whole swarms, each seen as an operator Φ. This report follows the information **up** those three scales, and states at every stage what is **served**, **real-but-disabled**, or a **stub**.

1 Executive summary

EFE aggregates information by **successive folding** across three scales. At each stage, a population reduces to a single vector, which becomes the unit of the stage above. It is elegant — and it must be described without embellishment.

Scale 1 — the nano-robot (nano-agent). Concretely, it is a **row of a latent tensor** `[N,384]` in bf16, plus a template tag (u8), an id and a salience (f32). It has **no behaviour, no autonomy, no local memory, no messaging**. What distinguishes agent i from agent j: its template (which *prior* it copies) and its deterministic random draw. It is real and served (efe-server Mode-1) — but it is a tensor row, not a robot.

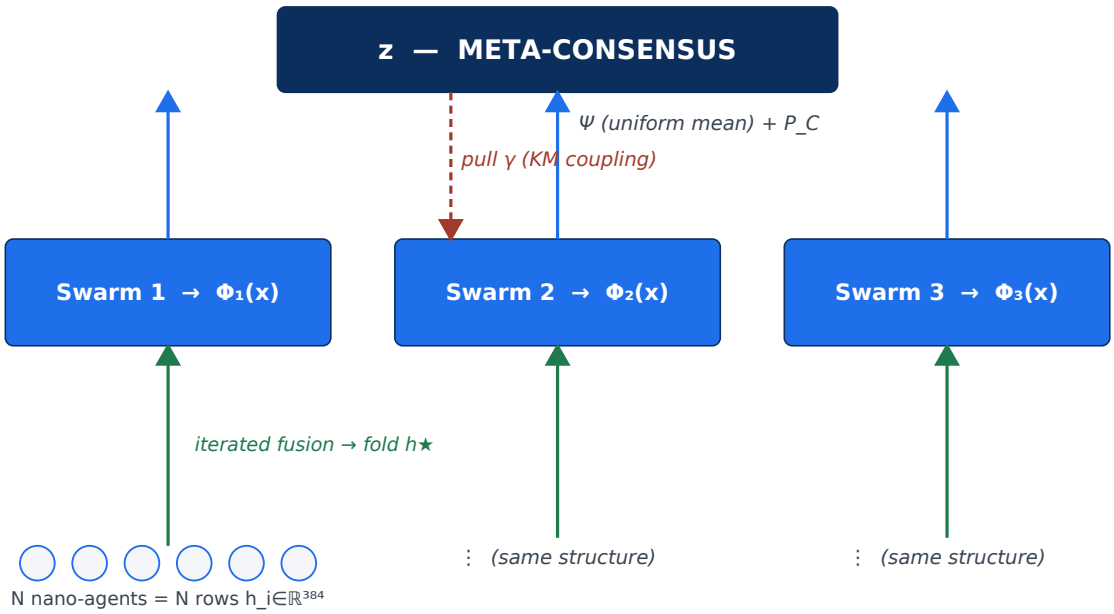
Scale 2 — the swarm. A `SwarmController` drives the population as a **linear state machine**: genesis → iterated fusion (N×N softmax attention) → consensus detection → alignment (projection onto a cone, with a breach gate that **aborts** the request) → dissolution into **one centroid** (the Fréchet mean). That centroid h★ **is** the swarm's answer. Real and served. But it is a fixed strategy pipeline — **not** emergent self-organisation — and convergence is best-effort at most (a round floor can delay, never guarantee, convergence).

Scale 3 — the meta-swarm. An **entire** swarm becomes an operator Φ; the controller couples M participants through a Krasnoselskii–Mann fixed point, circulating a meta-consensus as the *pull*. The gRPC/mTLS transport and the coupled iteration are served — but the **default** participant is a **mock** $\Phi(x)=\kappa\cdot x$, aggregated by a uniform mean with **no robustness whatsoever**. The real efe-core participant (a swarm served as Φ), the H1 Lipschitz probe and the **measured** bounds (not proven: $\hat{L}\approx 0.37$ fixed / 0.69 dynamic, refuting the analytic certificate $\beta_1\approx 0.17$) all live behind the `core-integration` feature, off by default. **This is the scale where all of our work lives.**

> **Three honesty retractions never to forget**
>
> (1) **There is no "safety nano-robot that vetoes harmful content".** Per-template behaviours (fusion kernel, consensus weight) are **not wired**; `NaSafety`'s weight-2 refusal vote is **not applied**, and `NaSafety` is not even in the served population. (2) The served template *priors* are **semantically empty** (`scale·sin(i)`, "Not meaningful") — the load-bearing signal is the injected *problem embedding* (a real sentence-transformer vector) + the catalog anchors, not the templates. (3) The population is **fixed at genesis**: no agent is created or killed at runtime on the served path.

2 The vertical principle — aggregation by folding

EFE's connecting thread is a **cascade of reductions**: N tensor rows fold into 1 swarm centroid; M swarm centroids fold into 1 meta-consensus. Each scale turns a *population* into a single *vector*, which becomes the atom of the scale above. The diagram below gives the overview; the following sections detail each stage, code in hand.



EFE's vertical. Bottom: nano-agents (rows of the latent tensor) exchange through N×N softmax attention and fold (iterated fusion → centroid h★) into a swarm's answer. Middle: each swarm, exposed as an operator Φ, is a participant of the meta-swarm. Top: the controller aggregates the Φ (uniform mean Ψ) and projects onto safety (P_C) to form the meta-consensus z, sent back to the participants as the pull γ (Krasnoselskii–Mann coupling).

Scale	"Agent" =	What flows up	Folding operator	Default status
1 · Nano-robot	tensor row h_i	one vector $\mathbb{R}^{384}$	softmax attention → aggregate	**SERVED**
2 · Swarm	folded population	centroid h★ (the answer)	Fréchet mean (extract_centroid)	**SERVED**
3 · Meta-swarm	swarm = Φ	meta-consensus z	Ψ uniform mean + projection P_C	**MOCK** κ·x

3 Scale 1 — the nano-robot (nano-agent)

Status: **PROD · served** (in efe-server Mode-1).

What a nano-agent IS. At genesis (`genesis/spawn_kernel.rs:107`), the population is a Structure-of-Arrays: a latent tensor `[N,D]` in bf16, a template vector `[N] u8`, the *priors* `[K,D]`, and a mask `[N] u8`. For each agent i: h_i = prior[template$_i$] + N(0,1)·σ (noise drawn from a per-swarm `Pcg64`), and mask[i]=1. The only per-agent metadata (`NanoAgentBatch`) are: the template, a `NanoAgentId`, and an f32 salience (initialised to 1.0, mutated by fusion). **There is no identity beyond a tensor row + a tag + a scalar**: no behaviour, no goal, no memory, no message.

What distinguishes one agent from another. Exactly three things: its **template index** (which *prior* row it copies), its **row** in the tensor, and its **random draw** (deterministic from the swarm seed and the position). The mask excludes an inactive row from attention; at genesis it is all ones. An agent's "voice" is its row, read by all the others at every round through attention.

"Death" = dissolution. There is no per-agent death event. The population "dissolves" when `extract_centroid` folds all active rows into **one consensus vector** (the weighted/Fréchet mean). It is a **reduction** at the tensor level (N rows → 1 vector), not N agents "reaching an agreement". The agents are only freed by RAII when the batch and the ELS are dropped.

The nano-scale guardrails (against over-selling)

• **Empty template priors** — the served registry (`StubTemplateRegistry`) produces `prior = scale·sin(i)`, explicitly "Not meaningful — just distinguishable". The load-bearing signal is the injected *problem embedding* (a real all-MiniLM-L6 vector, kept dominant, β_0=0.45) + the catalog anchors. • **Per-template behaviours not wired** — `NanoAgentTemplate` declares `fusion_kernel/consensus_weight/centroid_init`, but only `default_share()` is consumed. `NaSafety`'s weight-2 refusal vote is **not applied**, and `NaSafety` is not in the MVP population (NaSem/NaLog/NaCrit only). **Do not present an operating safety nano-agent.** • **Fixed population** — `respawn_diversify` is a no-op; mask-based deactivation (`collapse::PartialRespawn`) exists but is **not wired**. No dynamic spawn/kill is served. • **CPU only** — the CUDA birth kernel is "Sprint 3+".

4 Scale 2 — the swarm

Status: **PROD · served** (SwarmController, the real Mode-1 engine).

A swarm is a `SwarmController` (`swarm/controller.rs:452`) that sees the world only through **five pluggable strategies** (`swarm/strategies.rs`): *Genesis* (birth), *Fusion* (the exchange), *ConsensusDetector* (when to stop — stateful, reset per request), *Dissolver* (extract the centroid + decode) and *AlignmentProjector* (the final safety gate). The set of stages is **fixed**; there is no reconfigurable graph.

4.1 · The state machine (drive loop)

The `drive()` loop (`controller.rs:452-634`) is a strictly linear pipeline: **(1) Genesis** → `(batch, els)`; **(2) Fusion**, for round 1..max_rounds: `run_round` then `detector.evaluate` — a *Converged* verdict before the `min_rounds` floor is **downgraded to Continue** (the D6 floor); *Divergence* → error; *ModeCollapse* → a recovery attempt (noise + respawn) then error; no convergence after the loop → `ConvergenceTimeout`; **(3) Dissolution**: `extract_centroid` → h★, then **alignment BEFORE decoding** with the breach gate; **(4) RAII cleanup**. The `SwarmState` enum (`Pending|Genesis|Fusion|Dissolution|Done|Failed`) is **observational** (stamped onto the tracing span) — a *documented* state machine, not yet a type-enforced one (typestate = Phase-2).

4.2 · The intra-swarm exchange and the fold

The exchange is the N×N softmax attention already described: each row rewrites itself as a consensus-weighted blend of all the others, then a β-mix toward the *problem embedding* and a projection onto the cone. The final fold (`extract_centroid`) is the (Fréchet) mean of the active rows: it is the swarm's answer, $h★ \in \mathbb{R}^{384}$. On the served path, `PassThroughDissolver` returns the centroid that the fusion round deposited in a shared slot (`LastCentroid`).

4.3 · Safety: the alignment cone and the breach gate

The alignment cone (`AlignmentCone{anchors, max_projection_distance}`) carries the "constitutional priors". The strict projector (`dissolution/alignment.rs`) computes the centroid's positive cosine weights against each anchor (negatives clamped to zero); if all are zero (orthogonal to every anchor) → **breach** (delta=∞); otherwise it blends `projected = (1-α)·raw + α·anchor_centroid` and, if `delta > radius·(1+tolerance)` → **breach**. The controller re-checks and, on breach, **aborts the whole request** (a breached centroid never reaches the decoder). It is a **hard failure**, not clamp-and-serve. Alignment is **required and executed on the served path** (`AlignmentAdapter::strict_default`, α=0.5, tolerance=0.05).

What the swarm is NOT

• **Not emergent / self-organising.** `drive()` is a fixed pipeline; the only adaptive element served is the attention dynamics. The word "emancipation" only names the **library's** fusion doctrine, which Mode-1 **deliberately tones down** (the input stays dominant: β_0 0.20→0.45, α_0 0.50→0.25). • **Convergence not guaranteed.** `min_rounds` is a *floor* that can delay or deny convergence, not guarantee it (`min_rounds > max_rounds` even guarantees the timeout). • **Untyped state machine** — enforced procedurally in the MVP.

5 Scale 3 — the meta-swarm

Status: transport **PROD · served** · real participant **real · off by default** · default = **mock only** (κ·x).

An entire swarm becomes an operator Φ. `SwarmController::phi_once(x)` (`controller.rs:412`) sets `spec.problem_embedding = x` and `spec.max_rounds = 1`, then runs a complete `execute()` (genesis → **exactly ONE** fusion round → dissolution → alignment gate → pass-through decode) and returns the raw centroid. So Φ(x): x → embedding → one round → centroid → back into x-space. **One round, NOT to convergence** — and it is not a lightweight kernel: every call pays the full genesis (arena allocation, spawn), dissolution and alignment. The coupled step wraps Φ in the Krasnoselskii–Mann mix: `x_new = (1-α-γ)·current + α·Φ(current) + γ·pull`.

The meta-swarm loop. `CoupledIteration::run` (`iteration/coupled.rs`) does, per step: (1) parallel *lift* of each participant's state into meta-space; (2) aggregation Ψ of the votes → z; (3) safety projection `meta_safety.project(z)`; (4) the *pull* sent back to each participant; (5) a parallel `step_with_pull` barrier; (6) the residual. It stops on residual ≤ tolerance (Converged) or max_iters. **Before** iterating, H1 is pre-checked for every participant and the loop bails out (`HypothesisCheckFailed`) if any certificate is rejected. Safety is projected **twice** per step (participant output + meta-consensus).

But the default participant is a mock. `core-integration` is disabled by default in both crates; without it, the only participant is `MockParticipant` (Φ(x)=x or κ·x), and the aggregator is `UniformMetaOp` (a mean, Lipschitz bound exactly 1, **zero** outlier robustness). The alternative `AttentionMetaOp` (softmax) is a stub explicitly "NOT validated against the convergence theorem", behind an off feature, whose bound $1+2\|q\|/\tau$ can **exceed 1** (expansive). There is **no** outlier rejection, **no** median/trimmed scheme, **no** learned weighting anywhere. The real efe-core bridge (a swarm served as Φ) only compiles with the feature; an efe-core manifest row on a default build **fails construction** (never a silent fallback to the mock).

H1: a falsifier, not a proof

Convergence rests on H1 (Φ non-expansive, $L_\Phi \le 1$). The Lipschitz probe (at boot, for efe-core engines) measures a **lower** bound $\hat{L}$ by adversarial black-box sampling: it can **reject** H1 ($\hat{L} \ge 1-\text{margin} \Rightarrow$ refuse to serve, *fail-closed*), never **certify** it. The fixed topology makes Φ continuous by construction (it removes the genesis-routing discontinuity), but the contraction remains **property-tested, NOT certified**. Measured: $\hat{L} \approx 0.37$ (fixed) / 0.69 (dynamic) — the analytic certificate $\beta_1 \approx 0.17$ is refuted.

6 The aggregation thread — one vector, end to end

To make the cascade concrete, let us follow a vector from a nano-agent row to the meta-consensus, stating at every hop what is real and what is not:

Hop	Transformation	Honest reality
Birth	h_i = prior[template] + noise·σ (one row $\mathbb{R}^{384}$)	Real, served. But the *prior* is distinguishable noise, not knowledge.
Intra-swarm exchange	N×N softmax attention → β-mix → cone projection (iterated)	Real, served. The injected *problem embedding* dominates (β_0=0.45).
Swarm fold	extract_centroid: N rows → 1 centroid h★	Real, served. Fréchet-mean reduction; no outlier rejection.
Safety gate	projection onto the cone; breach ⇒ request aborted	Real, served, *fail-closed*. A hard failure, not clamping.
Swarm → Φ	phi_once: h★ becomes one step of the operator Φ(x)	Real but **off by default** (core-integration). One round, not converged.
Meta aggregation	Ψ = uniform mean of the Φ_k + projection P_C → z	Transport served; by default the Φ_k are **mocks** κ·x. No robustness.
KM coupling	z sent back as the pull γ; x_new = mix(current, Φ, pull)	Real, served. H1 pre-checked; wire certificates = stub (checked at boot only).

The honest lesson of this table: **the aggregation machinery is real and coherent end to end**, but the *content* flowing up is thin by default (empty priors, mock participants), and the only scale genuinely served with load-bearing content is the single swarm in efe-server, where the real *problem embedding* dominates.

7 What we built — making a real swarm servable as Φ (M1)

Our work targets scale 3: replacing the κ·x mock with a **real efe-core swarm** as the meta-swarm's operator Φ, served over gRPC, every step locked fail-closed. Seven commits (8fcb3dc → 665ead0), July 13–14, 2026.

Commit	Milestone	Delivered (scale 3)
8fcb3dc	M1.a — the seam	The meta step calls a real `SwarmController` via `phi_once`; `step_with_pull` = the KM coupled mix.
720d07c	M1.b — deterministic Φ	`init_sigma=0` + seed derived from the spec id → bit-reproducible Φ (the probe's prerequisite).
a8f1798	M1.c — H1 probe	Adversarial Lipschitz estimator (discontinuity bisection), empirical certificate, strict fail-closed gate.
c53a7f6	M1.d — SERVED	efe-core bridge wired into the gRPC layer (engine block, config_hash, boot gate, require_convergence). Default = mock.
b9a8fce	Palier-2 — fixed topology	Φ continuous by construction. The measurement: $\hat{L}$ 0.37 (fixed) / 0.69 (dynamic) — $\beta_1 \approx 0.17$ refuted as a certificate.
665ead0	Review debt	10 findings closed: an engine-identity reload bypass bug + over-sell retractions (the confounded ~1.9 ratio, over-assertive docstrings).

Verified before commit (665ead0)

Default suite: **20 binaries, 0 failures** (lib 368). Feature suite `core-integration`: **27 binaries, 0 failures** (efe-core e2e over gRPC 5/5). Adversarial re-review, 4 lenses: **0 confirmed findings**.

8 The honesty ledger (all three scales)

Tempting claim	Reality, verified in the code
“Autonomous nano-robots collaborate.”	**No.** A nano-agent is a tensor row + a tag + a salience. No behaviour, no messages. The collaboration = attention (a matmul).
“A safety nano-agent vetoes harmful content.”	**Not wired.** `NaSafety`’s weight-2 refusal vote is not applied; `NaSafety` is not in the served population.
“The swarm self-organises (emergence).”	**No.** A fixed strategy pipeline. Mode-1 **tones down** the emancipation doctrine (input dominant).
“Agents are born and die dynamically.”	**No.** The population is fixed at genesis. `respawn_diversify` is a no-op; `PartialRespawn` is not wired.
“Guaranteed / proven convergence.”	**Best-effort.** Failure = `ConvergenceTimeout`. H1 = a falsifier (lower bound $\hat{L}$), never proven; $\beta_1 \approx 0.17$ refuted (0.37/0.69).
“The meta-swarm runs real swarms by default.”	**Mock by default** ($\kappa \cdot x$). The real swarm-as-Φ is off (`core-integration`) — that is the M1 work.
“Robust / weighted aggregation across swarms.”	**Uniform mean only** (`UniformMetaOp`). One adversarial participant moves the consensus.
“Template priors encode knowledge.”	**No.** `scale·sin(i)`, “Not meaningful”. The knowledge comes from the injected *problem embedding* + the anchors.

To be **credited** (understating would be just as dishonest): the `drive()` state machine with its decoupled strategies and its **alignment breach gate that aborts the request**; the D6 convergence floor + the tightened consensus; the determinism engineering of Φ (which makes the probe possible); the meta-swarm’s coupled iteration with double safety projection, the *fail-closed* H1 pre-check, and the strict `ConvergenceFailed`; and the fail-closed efe-core manifest bridge (an efe-core row without the feature fails — it never silently falls back to the mock). The **honesty machinery** is real, even where the content is thin.

9 What remains — an honest roadmap (per scale)

- **Scale 1** — make the templates meaningful: wire `fusion_kernel/consensus_weight` (including the safety refusal vote) and replace the `sin(i)` priors with real priors; otherwise stop describing “specialised nano-robots” and speak of a homogeneous population driven by the embedding.
- **Scale 2** — move the state machine to *typestate* (Phase-2); wire a real mode-collapse recovery (a no-op today) or retract the self-healing claim; outlier-robust centroid weighting (`DistanceInverse`, “Sprint 8+”).
- **Scale 3** — serve the real participant by default (⇒ certificates **on the wire**, milestone M4, so strict mode does not brick the served path); a served robust aggregation (median/trimmed); reconcile the $\beta_1 \approx 0.17$ vs $\hat{L} \approx 0.37$ gap (stage-by-stage measurement) and re-measure fixed-vs-dynamic on a **matched population** (the confound).
- **Cross-cutting** — land the “pure Φ” fix cleanly (uncommitted) and honour purity under concurrency for the default Fixed participant (today pure only under single-flight execution).

10 Annex — served vs library defaults, and index

The served profile (efe-server Mode-1) tightens the library

Knob	Library default	Served Mode-1	Effect
`min_rounds`	0 (no floor)	4	convergence floor (D6)
consensus (conf./hyst.)	0.50 / 2	0.65 / 3	a stricter consensus
fusion β_0 / α_0	0.20 / 0.50 (emancipation)	0.45 / 0.25	input stays dominant
`init_sigma`	0.02 (request) / 0.0 (latent)	same	deterministic Φ on the latent path
meta participant	—	Mock κ·x; efe-core off	default = mock

Key crates of the vertical

- **Nano + swarm**: `efe-core` (`swarm/controller.rs`, `genesis/`, `fusion/`, `els/`, `dissolution/`); served via `efe-server/mode1.rs`.
- **Meta-swarm**: `efe-meta-swarm` (operators, coupled iteration, H1 probe) + `efe-meta-swarm-server` (served participant, gRPC/mTLS, manifest, reload guard).
- **Tests**: nano/swarm ≈ 427 inline (grep); meta-swarm — the M1 suites **re-run this session** (default 368 lib + 20 bins, feature 27 bins, 0 failures).

> **Method note & final reservation**
>
> This report is grounded on a surgical code reading (file:line citations) and an honesty review. The nano/swarm test counts are **static grep estimates**; only the M1 suites (scale 3) were re-run this session. “Nano-robot” is presentation vocabulary; the code says “nano-agent” — a tensor row. When in doubt, this document always chooses the lower formulation.